

Wunomo AI Backend Startup Guide

Executive Summary

This startup guide is the operational handbook for launching, configuring, running, validating, and maintaining the **Wunomo AI DataOps Agent backend**. It is written in the same structured format as the main product documentation, but focused entirely on practical setup and usage.

It covers environment setup, system requirements, folder expectations, Docker services, startup commands, testing commands, development workflow, common run sequences, and operational notes that help a developer or team get the backend running from zero.

Product Context

Item	Description
Product	Wunomo AI
Personality Covered	1. DataOps Agent
Guide Type	Backend startup and operations guide
Primary Goal	Help a developer start, test, and work with the backend from local development to service-level operation
Backend Style	FastAPI + PostgreSQL + Redis + Celery + AI orchestration

What This Guide Covers

Area	Included
Local startup flow	Yes
Docker-based startup	Yes
Core commands	Yes
Test commands	Yes
Environment variables	Yes
Service overview	Yes
Developer workflow	Yes
Common troubleshooting orientation	Yes

Backend Stack

Layer	Technology
API Framework	FastAPI
Agent Layer	LangGraph + LangChain
Primary Database	PostgreSQL 16
ORM / DB Access	SQLAlchemy async + asyncpg
Cache / Queue	Redis

Layer	Technology
Async Jobs	Celery
LLM Layer	Gemini primary path, fallback LLM path described in project context
Auth	JWT with python-jose, bcrypt, passlib
Testing	pytest + pytest-asyncio
Logging	structlog
Container Runtime	Docker Compose

Expected Project Layout

The earlier build context identifies the working project path as follows:

Item	Value
Local Project Path	`C:\Personal Projects\workforce-agent`
Backend App Layout	`dataops-agent/backend` style modular Python backend described in project context
Important Root Files	`main.py`, `config.py`, `database.py`, `docker-compose.yml`, `requirements.txt`, `.env.example`, `README.md`

Core Services

The backend is designed to run as a multi-service stack.

Service	Purpose	Default Port
Backend API	Main FastAPI application	`8000`
PostgreSQL	Primary persistent database	`5432`
Redis	Queue and async support	`6379`
Celery Worker	Background task execution	Internal service
Celery Beat	Scheduled background jobs	Internal service

Prerequisites

Before starting the backend, make sure the following are available on the system.

Requirement	Why It Is Needed
Docker	To run the full backend stack with containers
Docker Compose	To manage multi-service startup
Python 3.11	For local development and test execution
Git	To clone and manage the repository
Valid API keys	For LLM-backed features

Environment Variables

The build context lists the following key environment variables.

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string
`REDIS_URL`	Redis connection string
`GEMINI_API_KEY`	Primary LLM access
`GROQ_API_KEY`	Fallback LLM access
`JWT_SECRET`	JWT token signing
`APP_ENV`	Environment mode such as development

Example values from project context

Variable	Example
`DATABASE_URL`	`postgresql+asyncpg://dataopsuser:changeme@postgres:5432/dataops`
`REDIS_URL`	`redis://redis:6379/0`
`APP_ENV`	`development`

First-Time Setup Flow

Use the following startup sequence when running the backend for the first time.

Step	Action
1	Open the project folder
2	Create or update the `.env` file from `.env.example`
3	Fill in database, Redis, JWT, and LLM variables
4	Start Docker services
5	Verify backend health
6	Run tests
7	Begin API usage or development work

Local Shell Navigation

Windows example

```
cd "C:\Personal Projects\workforce-agent"
```

If backend work is inside a nested project folder

```
cd "C:\Personal Projects\workforce-agent"
```

If the codebase uses a nested backend folder in your machine, move into that exact backend root before running commands.

Environment File Setup

Create `.env` from example

```
copy .env.example .env
```

If you are using PowerShell:

```
Copy-Item .env.example .env
```

Fill required values

Make sure your `.env` contains the following logical fields:

```
DATABASE_URL=postgresql+asyncpg://dataopsuser:changeme@postgres:5432/dataops<br/>REDIS_URL=redis://redis:6379/0<br/>GEMINI_API_KEY=your_key_here<br/>GROQ_API_KEY=your_key_here<br/>JWT_SECRET=your_secret_here<br/>APP_ENV=development
```

Main Startup Commands

Start the full backend stack

```
docker compose up --build
```

This starts the backend API, PostgreSQL, Redis, Celery worker, and Celery beat together.

Start in detached mode

```
docker compose up -d --build
```

Stop the stack

```
docker compose down
```

Stop the stack and remove volumes if needed

```
docker compose down -v
```

Use the volume removal version only when you intentionally want a clean reset.

Common Docker Operations

Command	Purpose
<code>`docker compose up --build`</code>	Build and start all services
<code>`docker compose up -d --build`</code>	Start all services in background mode
<code>`docker compose down`</code>	Stop all running services
<code>`docker compose down -v`</code>	Stop services and remove volumes
<code>`docker compose restart backend`</code>	Restart only the API container
<code>`docker compose logs -f backend`</code>	Watch backend logs live
<code>`docker compose logs -f celery-worker`</code>	Watch worker logs live

Command	Purpose
<code>`docker compose logs -f celery-beat`</code>	Watch scheduler logs live
<code>`docker compose ps`</code>	Show current service status

Useful Runtime Commands

Restart backend only

```
docker compose restart backend
```

View backend logs

```
docker compose logs -f backend
```

View all service states

```
docker compose ps
```

Open a shell inside backend container

```
docker compose exec backend sh
```

If ``bash`` is available in your container:

```
docker compose exec backend bash
```

API Access

Once the backend is running, the main API should be available through the FastAPI app on port ``8000``.

Endpoint Area	Purpose
<code>`/health`</code>	Service health check
<code>`/docs`</code>	Interactive API docs in development
<code>`/api/v1/auth/*`</code>	Auth flows
<code>`/api/v1/sources/*`</code>	Source management
<code>`/api/v1/pipelines/*`</code>	Pipeline operations
<code>`/api/v1/quality/*`</code>	Quality rules
<code>`/api/v1/incidents/*`</code>	Incident workflows
<code>`/api/v1/analytics/*`</code>	Metrics and analytics
<code>`/api/v1/approvals/*`</code>	Approval workflows
<code>`/api/v1/transformations/*`</code>	Transformation generation and execution

Open API docs

```
http://localhost:8000/docs
```

Check health endpoint

```
http://localhost:8000/health
```

Test Commands

The earlier run logs show the backend tests were being executed from inside the backend container with pytest.

Run API tests

```
docker compose exec backend pytest tests/test_api.py -v --tb=short
```

If your test file uses the earlier project naming convention

```
docker compose exec backend pytest tests/testapi.py -v --tb=short
```

Run all tests

```
docker compose exec backend pytest -v
```

Run a specific test

```
docker compose exec backend pytest tests/test_api.py::test_health -v
```

Validation Workflow After Startup

Use this sequence after the stack comes up.

Step	Validation Action	Expected Outcome
1	Check `docker compose ps`	All required services show as running
2	Open `/health`	Backend responds successfully
3	Open `/docs`	FastAPI docs load in browser
4	Run auth tests	Auth routes behave correctly
5	Run API tests	Core routes validate successfully
6	Review logs	No boot errors or missing env issues

Typical Development Workflow

A normal development session can follow this pattern.

Step	Command / Action	Purpose
1	`docker compose up -d --build`	Start all services
2	`docker compose logs -f backend`	Watch startup logs
3	Edit code locally	Build or modify backend files

Step	Command / Action	Purpose
4	`docker compose restart backend`	Reload service after changes
5	`docker compose exec backend pytest -v`	Validate correctness
6	Open `/docs`	Manually inspect API shape

Common Startup Sequences

Quick start sequence

```
cd "C:\Personal Projects\workforce-agent"<br/>copy .env.example .env<br/>docker compose up -d --build<br/>docker compose ps<br/>docker compose logs -f backend
```

Restart after code updates

```
docker compose restart backend<br/>docker compose logs -f backend
```

Run tests after restart

```
docker compose exec backend pytest -v
```

Suggested First Checks Inside the Product

After the backend starts, the first operational checks should usually be:

1. Register a user
2. Log in and obtain a token
3. Open `/docs`
4. Create or inspect a source
5. Run schema profiling
6. Create a pipeline
7. Add a quality rule
8. Trigger a pipeline run
9. Inspect results
10. Test transformation generation endpoints

Common Router Areas to Verify

Router	Why Verify It Early
Auth	Confirms base security and session logic
Sources	Confirms DB persistence and source operations
Pipelines	Confirms orchestration routes
Quality	Confirms rule engine integration
Incidents	Confirms issue-tracking path

Router	Why Verify It Early
Transformations	Confirms Phase 4 backend execution path
Analytics / Approvals	Confirms supporting operational routes

Command Reference

Docker

```
docker compose up --build<br/>docker compose up -d --build<br/>docker compose down<br/>docker
compose down -v<br/>docker compose ps<br/>docker compose restart backend<br/>docker compose logs -f
backend<br/>docker compose logs -f celery-worker<br/>docker compose logs -f celery-beat<br/>docker
compose exec backend sh
```

Pytest

```
docker compose exec backend pytest -v<br/>docker compose exec backend pytest tests/test_api.py -v
--tb=short<br/>docker compose exec backend pytest tests/testapi.py -v --tb=short<br/>docker compose
exec backend pytest tests/test_api.py::test_health -v
```

Windows file preparation

```
copy .env.example .env<br/>Copy-Item .env.example .env
```

Troubleshooting Orientation

Symptom	Likely Area to Check
Backend does not start	`env` values, build errors, missing dependency
`docs` not loading	Backend container status, port mapping, startup logs
DB-related errors	`DATABASE_URL`, Postgres container, migration / connection setup
Async task issues	Redis status, Celery worker logs
Auth failures	JWT secret, route config, test setup
AI feature failures	LLM API keys or model service configuration
Failing tests	Recent code changes, DB state, event loop or middleware issues

Development Rules to Preserve

The earlier backend context established a few coding and operational rules that should continue to be respected during startup and development work.

Rule	Description
UTC handling	Use timezone-aware `datetime.now(timezone.utc).replace(tzinfo=None)` pattern instead of `datetime.utcnow()`
Tenant isolation	All queries must filter by `tenant_id`
Async DB use	Use async DB session patterns consistently
Module error pattern	Modules return error dictionaries, routers convert to HTTP exceptions
Structured logging	Use structlog throughout the backend

Operational Best Practice

For day-to-day work, the safest and cleanest way to run the backend is:

- Keep the stack running with Docker Compose
- Make code changes locally
- Restart only the affected service when possible
- Run tests after meaningful changes
- Check `/docs` and /health` regularly`
- Keep `.env` values valid and separated by environment`
- Avoid wiping volumes unless you intentionally want a reset

Recommended Team Usage

Team Member	How They Should Use This Guide
Backend Developer	Use it for local startup, restart, testing, and debugging
New Team Member	Use it as onboarding documentation
QA Engineer	Use it for validation and route testing
DevOps / Platform Engineer	Use it to understand service shape and dependencies
Founder / Product Lead	Use it to understand what is required to launch the backend locally

Final Note

This startup guide is meant to be the practical companion to the main Wunomo AI backend documentation. The main product document explains what the system is and why it matters. This guide explains how to run it, validate it, and work with it confidently.

As Wunomo AI grows and new personalities are added, this same structured startup format can be reused for every future backend personality as well.